Sprint 07 - Appointment Booking System - Module Development Specification

Stage PFE – Entreprise **VOID**

Stagiaire : Hibat Allah Rguiti

6 avril 2026

1. Introduction et Initialisation

Le module **Appointment** a été développé dans le but de fournir un système complet de gestion de rendez-vous sous Drupal. Ce système permet aux utilisateurs de réserver des rendez-vous avec des agences et des conseillers de manière interactive.

Le module a été initialisé à l'aide de la commande suivante :

```
drush generate module
```

Cette commande a permis de générer la structure de base du module, incluant les fichiers de configuration essentiels.

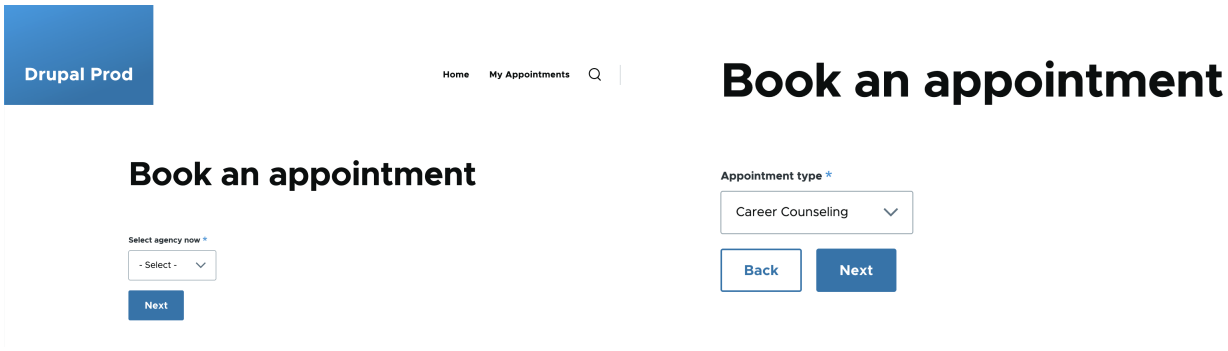
2. Démonstration

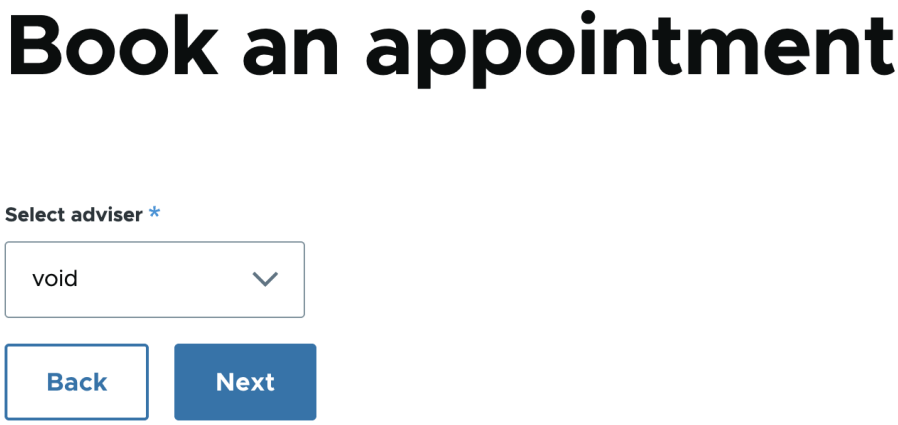
Cette section présente une démonstration complète du fonctionnement du module **Appointment**, en mettant en évidence les différentes étapes du parcours utilisateur ainsi que les fonctionnalités côté administration.

2.1. Formulaire de Réservation (Booking Form)

Le processus de réservation commence par un formulaire multi-étapes accessible via l'interface utilisateur.

— Sélection du type de rendez-vous
— Choix de l'agence
— Sélection du conseiller





2.2. Sélection de la Date via le Calendrier

Un calendrier interactif (FullCalendar) est intégré pour permettre à l'utilisateur de choisir un créneau disponible.

— Affichage des disponibilités

— Blocage des dates déjà réservées

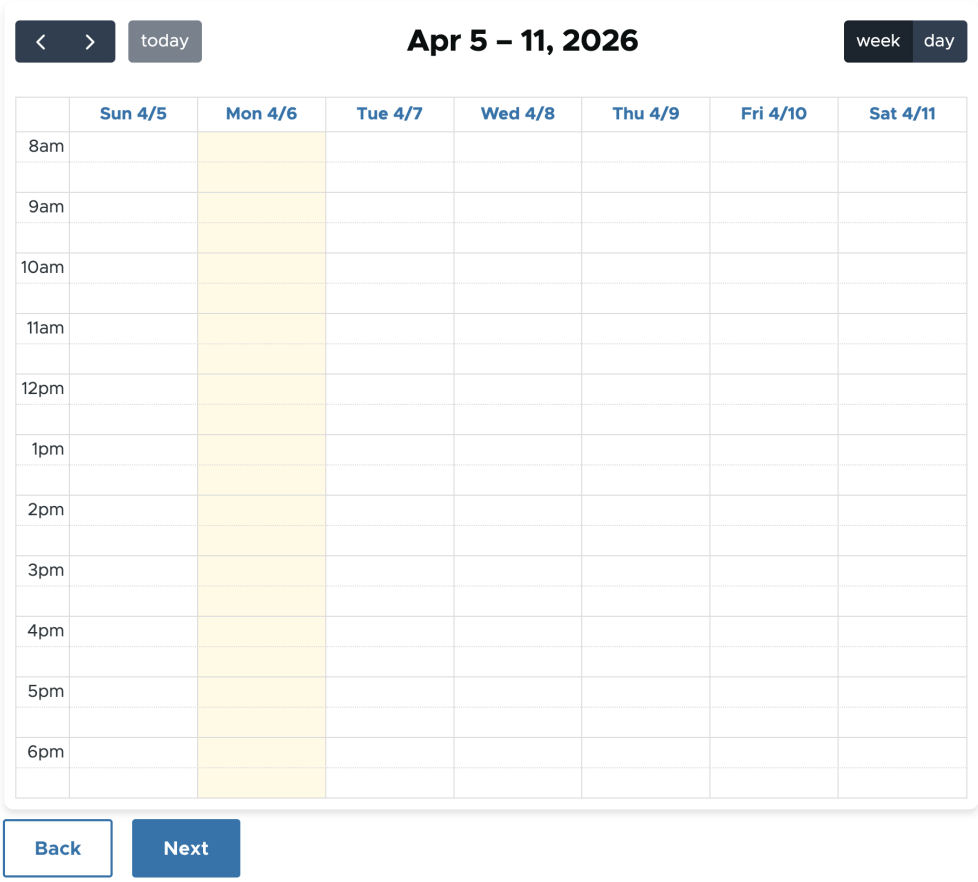


FIGURE 1 – Calendrier interactif

En cas de sélection invalide (date passée ou indisponible), un message d'erreur est affiché :

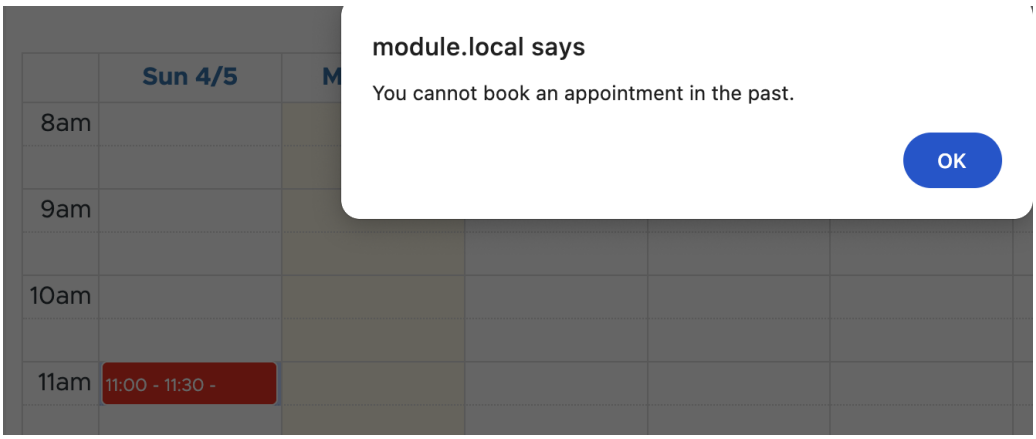


FIGURE 2 – Validation de la date

2.3. Informations Personnelles et Confirmation

L'utilisateur saisit ensuite ses informations personnelles (Nom, Email, Téléphone)

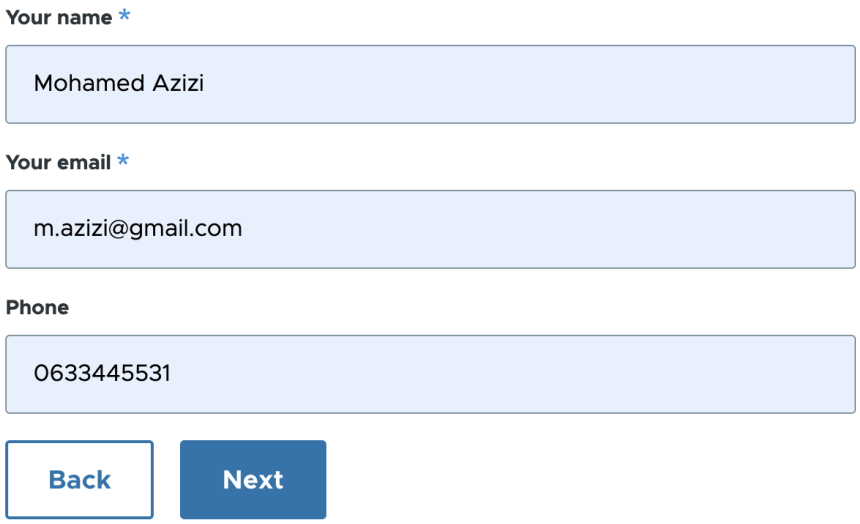


FIGURE 3 – Formulaire des informations personnelles

Une page de confirmation permet de vérifier les données avant validation finale :

2.4. Envoi de Mail

Après validation, un email de confirmation est envoyé à l'utilisateur (Test réalisé via MailHog).

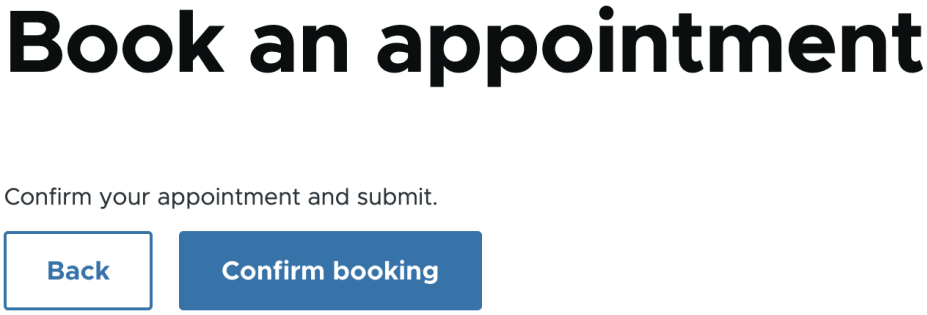
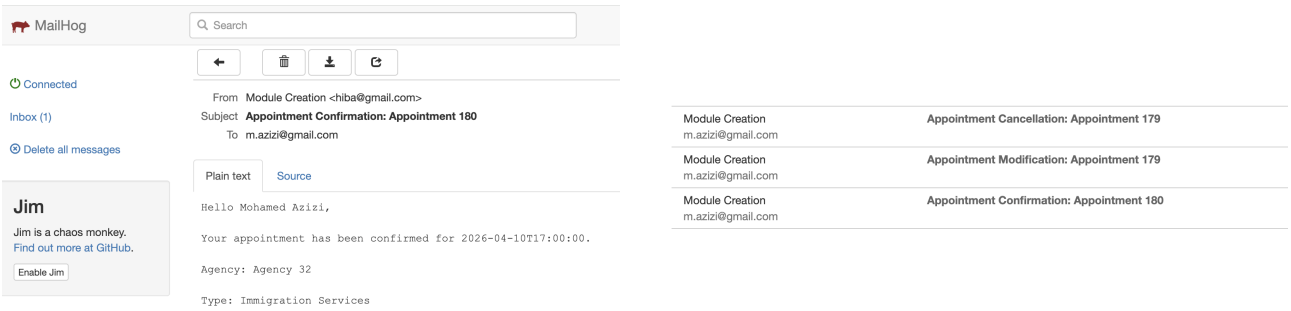


FIGURE 4 – Page de confirmation



2.5. Cas d'un Utilisateur Anonyme

Le module permet à un utilisateur non connecté de :

— effectuer une réservation,

— consulter ses rendez-vous via son email.

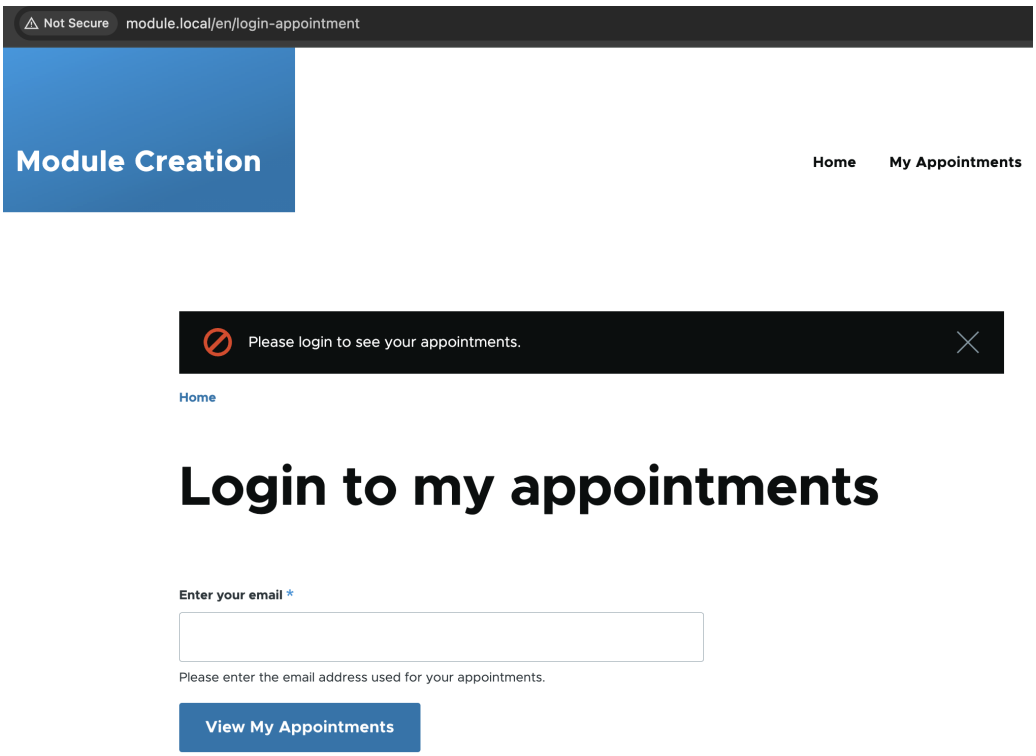


FIGURE 5 – Réservation en tant qu'utilisateur anonyme

2.6. Consultation des Rendez-vous

L'utilisateur peut consulter ses rendez-vous via la page **My Appointments** .De plus, chaque rendez-vous peut être consulté en détail :

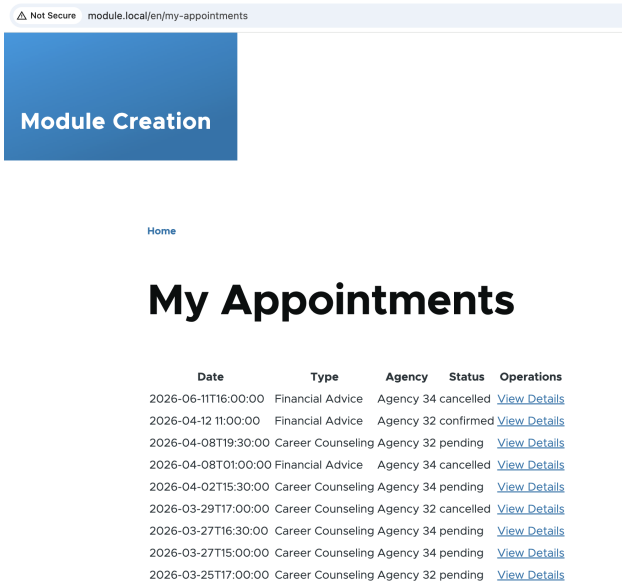


FIGURE 6 – Liste des rendez-vous

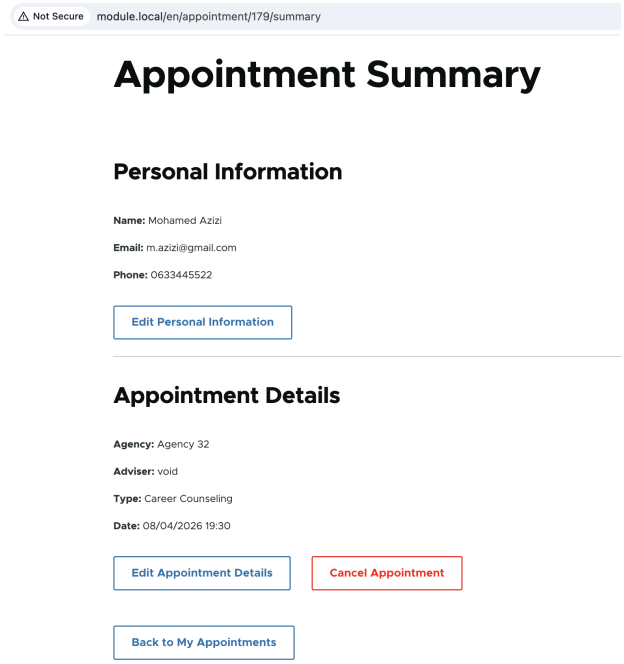


FIGURE 7 – Détail d'un rendez-vous

2.7. Vérification et Modification

Pour modifier ou annuler un rendez-vous, une vérification est requise :

— saisie du numéro de téléphone

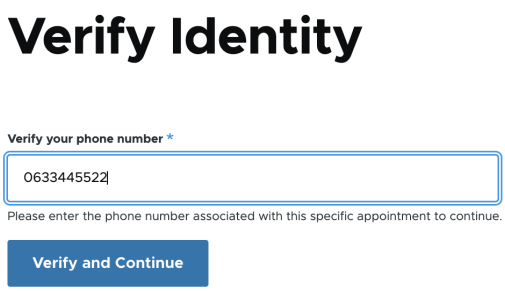


FIGURE 8 – Vérification par téléphone

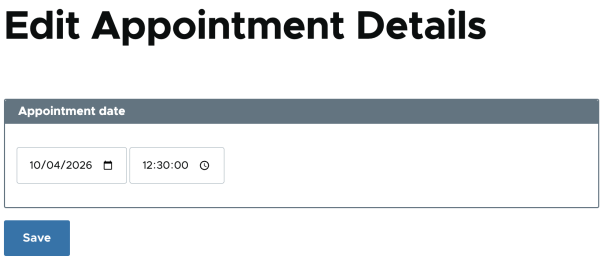


FIGURE 9 – Modification de la date

2.8. Interface Administrateur

Les administrateurs disposent d'une interface dédiée :

— visualisation de tous les rendez-vous

— filtres avancés

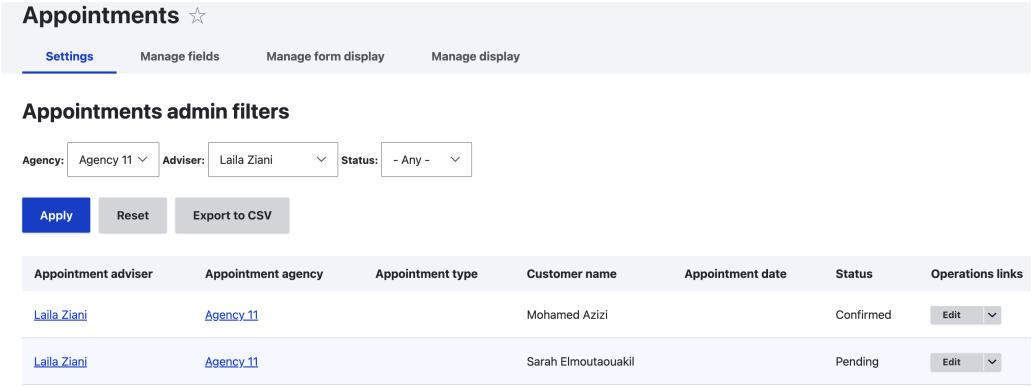


FIGURE 10 – Interface d'administration des rendez-vous

2.9. Export CSV

Un bouton permet d'exporter les données filtrées en fichier CSV.

— récupération des filtres

— génération dynamique du fichier

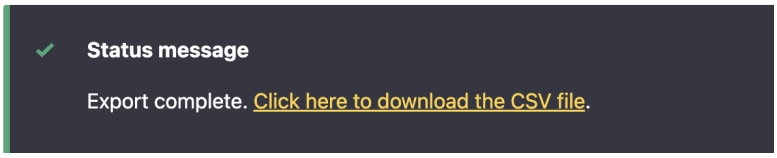


FIGURE 11 – Export des données en CSV

3. Architecture et Gestion des Assets

L'architecture du module repose sur une séparation claire entre :

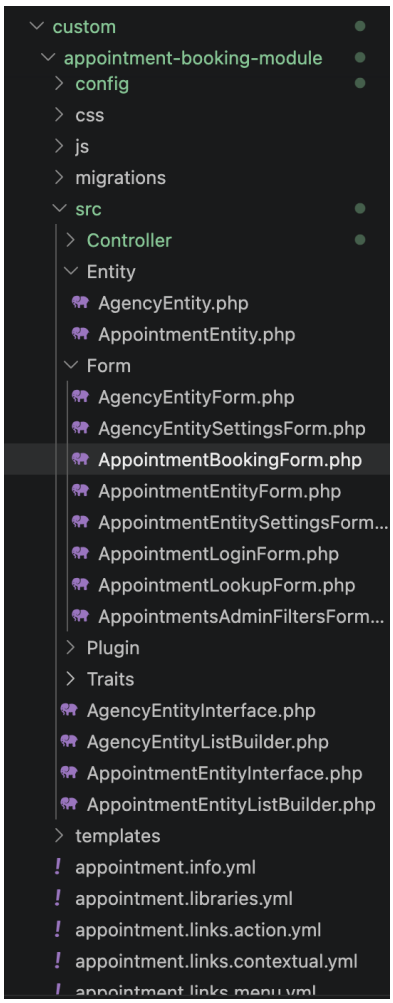
— la configuration (fichiers YAML),
— la logique métier (PHP),
— l'interface utilisateur (JavaScript et CSS).

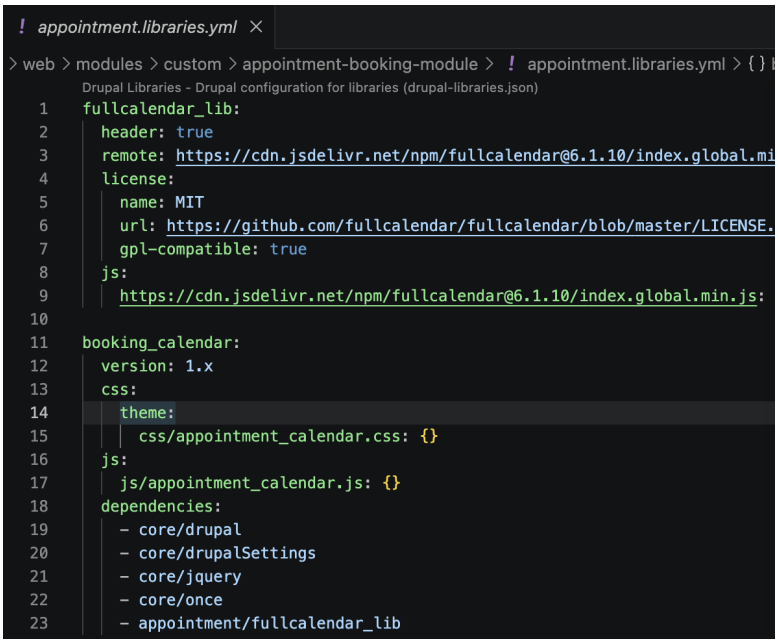
3.1. Librairies

Le fichier **appointment.libraries.yml** permet de déclarer les ressources nécessaires :

— Intégration de la bibliothèque **FullCalendar**.
— Définition d'une librairie personnalisée incluant :
  — fichier CSS pour le style du calendrier,
  — fichier JavaScript pour l'interaction avec le backend.

Le fichier JavaScript initialise le calendrier, affiche les rendez-vous existants et permet à l'utilisateur de sélectionner une date valide.





4. Entités de Contenu Personnalisées

Deux entités principales ont été développées :

— Entité Agency : Cette entité représente les agences disponibles.

— Entité Appointment : Cette entité représente les rendez-vous.

5. Formulaire de Réservation Multi-Étapes

Le formulaire principal est accessible via la route :

`/book-appointment`

Il est implémenté sous forme de formulaire multi-étapes :

— Chaque étape correspond à une partie des données (agence, conseiller, date, informations personnelles).

— Une variable **$step** est utilisée pour contrôler l'affichage.

```php
switch ($step) {

  case 1:
    $form['agency'] = [
      '#type' => 'select',
      '#title' => $this->t('Select agency'),
      '#options' => $this->getAgencyOptions(),
      '#empty_option' => $this->t('- Select -'),
      '#required' => TRUE,
      '#ajax' => [
        'callback' => '::updateAdviserOptions',
        'wrapper' => 'adviser-wrapper',
        'event' => 'change',
      ],
    ];
    break;

  case 2:
    $form['appointment_type'] = [
      '#type' => 'select',
      '#title' => $this->t('Appointment type'),
      '#options' => $this->getAppointmentTypeOptions(),
      '#empty_option' => $this->t('- Select -'),
      '#required' => TRUE,
    ];
    break;
```

5.0.1 Stockage Temporaire

Le module utilise **PrivateTempStore** :

— Stockage dans un espace nommé **appointment_booking**.

— Permet de conserver les données entre les étapes.

```php
public function __construct(PrivateTempStoreFactory $temp_store_factory)
{
  $this->tempStore = $temp_store_factory->get('appointment_booking');
}

public static function create(ContainerInterface $container)
{
  return new static(
    $container->get('tempstore.private')
  );
}
```

5.0.2 Validation

Plusieurs validations sont mises en place :

— la date ne doit pas être dans le passé,

— vérification des conflits de rendez-vous,

— respect des disponibilités.

```php
if ($step === 4) {
  }

  $date = new DrupalDateTime($date_val);
  if (!$date) {
    $form_state->setErrorByName(self::DATE_ERROR_NAME, $this->t('Please select a valid appointment date and time.'));
    return;
  }

  if (!$this->validateDateNotInPast($form_state, $date, self::DATE_ERROR_NAME)) {
    return;
  }

  $agency_id = (string) ($this->tempStore->get('selected_agency') ?? '');
  $adviser_id = (string) ($this->tempStore->get('selected_adviser') ?? '');

  $this->validateWithinAgencyHours($form_state, $agency_id, $date, self::DATE_ERROR_NAME);
  $this->validateWithinAdviserHours($form_state, $adviser_id, $date, self::DATE_ERROR_NAME);
  if ($adviser_id !== '') {
    $this->validateNoDoubleBooking($form_state, $agency_id, $adviser_id, $date, self::DATE_ERROR_NAME);
  }
}
```

5.0.3 Finalisation

Une fois toutes les étapes validées :

— création de l'entité Appointment,

— envoi d'un email de confirmation,

— redirection vers la liste des rendez-vous.

```php
try {
  $appointment->save();
  $this->messenger()->addMessage($this->t('Appointment successfully booked. ID: @id', ['@id' => $appointment->id()]));

  $email = $data['customer_email'] ?? '';
  $phone = $data['customer_phone'] ?? '';

  // Send confirmation email.
  if ($email !== '') {
    $mail_manager = \Drupal::service('plugin.manager.mail');
    $langcode = 'en';
    $params = [
      'title' => $appointment->label(),
      'name' => $appointment->get('field_customer_name')->value,
      'date' => $appointment->get('field_appointment_date')->value,
      'agency' => $appointment->get('field_appointment_agency')->entity ? $appointment->get('field_appointment_agency')->entity->label() : '',
      'type' => $appointment->get('field_appointment_type')->entity ? $appointment->get('field_appointment_type')->entity->label() : '',
    ];
    $mail_manager->mail('appointment', 'appointment_confirmation', $email, $langcode, $params, NULL, TRUE);
  }

  for ($i = 1; $i <= 5; $i++) {
    $this->tempStore->delete('step_' . $i);
  }

  // Store the verified email in session so the user can access their list.
  if ($email !== '') {
    $this->tempStore->set('verified_email', (string) $email);
  }

  $form_state->setRedirect('appointment.my_appointments');
} catch (\Exception $e) {
  $this->messenger()->addError($this->t('Error saving appointment: @msg', ['@msg' => $e->getMessage()]));
}
```

6. Espace Utilisateur et Sécurité

6.1. AppointmentSummaryController

Ce contrôleur gère l'affichage des rendez-vous.

6.1.1 Liste

Les rendez-vous sont récupérés selon :

— l'utilisateur connecté,

— ou l'email stocké en session, s'il s'agit d'un utilisateur anonyme qui vient de prendre un rendez-vous.

```php
public function list()
{
  $current_user = \Drupal::currentUser();
  $verified_email = '';

  // 1. Try to get email from logged-in Drupal user.
  if ($current_user->isAuthenticated()) {
    $verified_email = $current_user->getEmail();
  }
  // 2. Fallback to email stored in session (for anonymous users who just booked or logged in).
  else {
    $verified_email = (string) $this->tempStore->get('verified_email');
  }

  if (!$verified_email) {
    $this->messenger()->addError($this->t('Please login to see your appointments.'));
    return $this->redirect('appointment.login');
  }
```

6.1.2 Affichage des Détails

Avant d'afficher un rendez-vous, plusieurs vérifications sont effectuées :

— correspondance de l'email.

— vérification du référent (navigation depuis la liste).

```php
public function show(AppointmentEntityInterface $appointment)
{
  // 1. Security: verify the email in session matches the appointment.
  $current_user = \Drupal::currentUser();
  $verified_email = $current_user->isAuthenticated() ? $current_user->getEmail() : $this->tempStore->get('verified_email');
  $appointment_email = (string) $appointment->get('field_customer_email')->value;

  if ($verified_email !== $appointment_email) {
    $this->messenger()->addError($this->t('Please login with your email to access this appointment.'));
    return $this->redirect('appointment.login');
  }
```

6.2. Lookup Form

Pour modifier ou supprimer un rendez-vous :

— l'utilisateur doit saisir son numéro de téléphone,

— cela permet de vérifier son identité.

```php
public function buildForm(array $form, FormStateInterface $form_state, $appointment = NULL)
{
  if (!$appointment) {
    throw new \Symfony\Component\HttpKernel\Exception\NotFoundHttpException();
  }

  $form_state->set('appointment_id', $appointment->id());
  $action = \Drupal::request()->query->get('action');
  if ($action) {
    $form_state->set('redirect_action', $action);
  }

  $form['phone'] = [
    '#type' => 'textfield',
    '#title' => $this->t('Verify your phone number'),
    '#description' => $this->t('Please enter the phone number associated with this specific appointment to continue.'),
    '#required' => TRUE,
  ];

  $form['actions'] = [
    '#type' => 'actions',
  ];

  $form['actions']['submit'] = [
    '#type' => 'submit',
    '#value' => $this->t('Verify and Continue'),
    '#button_type' => 'primary',
  ];

  return $form;
}
```

7. Administration et Reporting

7.1. Vue Administrative

Une vue Drupal permet de gérer les rendez-vous :

— affichage sous forme de tableau,

— ajout de filtres (agence, conseiller, statut).

7.2. Export CSV

Le contrôleur **AppointmentExportController** permet :

— de récupérer les données filtrées,

— de générer un fichier CSV à la volée.

```php
public function start(Request $request)
{
  $filters = [
    'agency' => $request->query->get('agency'),
    'adviser' => $request->query->get('adviser'),
    'status' => $request->query->get('appointment_status'),
  ];

  $batch = [
    'title' => $this->t('Exporting Appointments to CSV...'),
    'operations' => [
      [[get_class($this), 'processBatch'], [$filters]],
    ],
    'finished' => [get_class($this), 'finishedBatch'],
  ];

  \batch_set($batch);
  return \batch_process(Url::fromRoute('entity.appointment.settings')->toString());
}
```

```php
public static function processBatch($filters, &$context)
{
  if (!isset($context['sandbox']['progress'])) {
    $context['sandbox']['progress'] = 0;
    $context['sandbox']['max'] = self::getCount($filters);

    // Create temporary file.
    $uri = 'temporary://appointments_export_' . time() . '.csv';
    $handle = fopen($uri, 'w');

    // Add CSV Header.
    fputcsv($handle, [
      'ID',
      'Title',
      'Customer Name',
      'Customer Email',
      'Date',
      'Agency',
      'Adviser',
      'Status'
    ]);

    fclose($handle);
    $context['results']['file_uri'] = $uri;
  }

  $limit = 50; // Process 50 rows per batch run.
  $ids = self::getAppointmentIds($filters, $context['sandbox']['progress'], $limit);
```

8. Migration des Données

Afin de faciliter les tests, j'ai inclus dans le module un système de migration basé sur des fichiers CSV (agencies.csv, advisers.csv, appointments.csv)

Les migrations sont définies dans des fichiers YAML.

9. Références et Ressources du Projet

Afin de faciliter l'accès au projet et permettre une meilleure compréhension de son implémentation, les ressources suivantes sont mises à disposition :

9.1. Repository GitHub

Le code source complet du module est disponible sur GitHub :

https://github.com/Hibat-Allah-RGUITI/appointment-booking-module

9.2. Vidéo de Démonstration

Une vidéo explicative présentant le fonctionnement du module ainsi que ses principales fonctionnalités est disponible via Loom :

https://www.loom.com/share/3113810b555d49cba5229b40b4e4c456

9.3. Conclusion Générale

Le projet réalisé met en œuvre un module Drupal complet intégrant des fonctionnalités avancées telles que la gestion des rendez-vous, un formulaire interactif multi-étapes, un système de validation robuste, ainsi qu'une interface d'administration avec export des données.